

DSAA Lab 01

Arrays and Dynamic Memory Allocation

Areeba (TCT-004)

Thar Institute of Engineering Sciences and Technology (TIEST)

CT-159: Data Structure Algorithms and Applications (Pr)

Engr. Aneel Kumar

October 30, 2025

Exercise

Q1: Write a code to copy data of a 2D array in a 1D array using Column Major Order.

Answer:

```
public class ColumnMajor {
    public static void main (String[] args){

        //2D Array
        int [][] matrix ={
            {1, 2, 3, 4},
            {5, 6, 7, 8},
            {9, 10, 11, 12}
        };

        int rows= matrix.length;
        int cols= matrix[0].length;

        //Creating 1D Array
        int[] flat = new int[rows*cols];

        int index=0;
        for(int c = 0; c< cols; c++){
            for(int r= 0; r < rows; r++ ) {
                flat[index++] = matrix[r][c];
            }
        }
        System.out.println("Column Major flattened array: ");
        for(int i = 0; i< flat.length; i++){
            System.out.print(flat[i] + (i == flat.length - 1 ? "\n" : ", "));
        }

    }
}
```

The screenshot shows an IDE window with the following components:

- Editor:** Displays the Java code for `ColumnMajor.java`, which is identical to the code provided in the previous block.
- Run Window:** Shows the execution output. The command executed is `C:\Users\Areeba\.jdk\valhalla-ea-23-valhalla+1-90\bin\java.exe -javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2.1\lib\idea_rt.jar=53`. The output is `Column Major flattened array: 1, 5, 9, 2, 6, 10, 3, 7, 11, 4, 8, 12`. Below the output, it states `Process finished with exit code 0`.
- Bottom Bar:** Shows the file path `DSA > src > ColumnMajor`, the Java version `32.1`, and the encoding `CRLF UTF-8`. The system tray at the bottom indicates `20°C Mostly cloudy` and the date `10/30/2025`.

Q2: Write a program to calculate the CGPA of students of all subjects of a single semester. Assume all the courses have the same credit hour (let's assume 3 credit hours). Data Structure

	Data Structure	Programming for AI	Digital Logic Design	Probability & Statistics	Finance & Accounting
Ali	3.66	3.33	4.0	3.0	2.66
Hiba	3.33	3.0	3.66	3.0	--
Asma	4.0	3.66	2.66	--	--
Zain	2.66	2.33	4.0	--	
Faisal	3.33	3.66	4.0	3.0	3.33

Answer:

```
public class CGPACalc {
    public static void main (String [] args ){

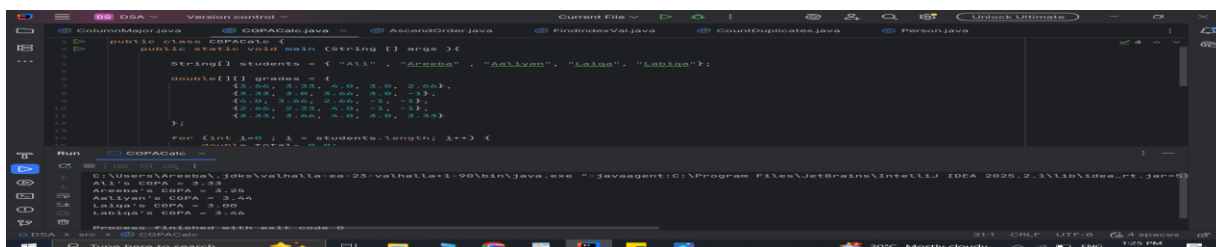
        String[] students = { "Ali" , "Areeba" , "Aaliyan", "Laiqa", "Labiqa"};

        double[][] grades = {
            {3.66, 3.33, 4.0, 3.0, 2.66},
            {3.33, 3.0, 3.66, 3.0, -1},
            {4.0, 3.66, 2.66, -1, -1},
            {2.66, 2.33, 4.0, -1, -1},
            {3.33, 3.66, 4.0, 3.0, 3.33}
        };

        for (int i=0 ; i < students.length; i++) {
            double total= 0.0;
            int count = 0 ;

            for(int j = 0; j< grades[i].length; j++ ){
                if(grades[i][j] != -1){
                    total += grades[i][j];
                    count++;
                }
            }
            double cgpa = (count ==0)? 0 : total/ count;
            System.out.printf("%s's CGPA = %.2f%n", students[i], cgpa);

        }
    }
}
```



Q3: Write a program to make given array in ascending order, do swapping without using third variable.
{8,9,7,5,3,4,1}

Answer:

```
public class AscendOrder {
    public static void main (String []args) {
        int[] arr = {8, 9, 7, 5, 3, 4, 1};
        int n = arr.length;

        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {

                    arr[j] = arr[j] + arr[j + 1];
                    arr[j + 1] = arr[j] - arr[j + 1];
                    arr[j] = arr[j] - arr[j + 1];

                }
            }
        }

        System.out.print("Array in Ascending Order: ");
        for (int num : arr) {
            System.out.print(num + " ");
        }
    }
}
```

The screenshot shows an IDE window with the file `AscendOrder.java` open. The code is the same as shown in the previous block. Below the code editor, the `Run` tab is active, showing the command used to execute the program: `C:\Users\Areeba\jdk\valhalla-ea-23-valhalla+1-90\bin\java.exe -javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2.1\lib\idea_rt.jar=53`. The output of the program is displayed as: `Array in Ascending Order: 1 3 4 5 7 8 9`. The process finished with exit code 0. The IDE interface also shows a sidebar with a file explorer and a bottom status bar with the text "DSA > src > AscendOrder > main".

Q4: (A) Find index number of 8, 55, 27.

0	1	2	3	4	5	6	7
34	10	66	27	47	8	55	78

(B) Find Value available on index number 4 & 6.

Answer:

```
public class FindindexVal {
    public static void main (String[] args){
        int[] arr = {34, 10, 66, 27, 47, 8, 55, 78};
        int [] searchValues = {8, 55, 27};

        System.out.println("(A) Index numbers: " );
        for (int value : searchValues){
            int index = -1;
            for (int i = 0 ; i < arr.length; i++){
                if(arr[i] == value){
                    index = i;
                    break;
                }
            }
            if (index != -1)
                System.out.println("Value " + value + " found at index " + index );
            else
                System.out.println(" Value " + value + " not found ");
        }
    }
}
```

```
System.out.println("\n(B) Values at given indexes: ");
int[] indexes = {4, 6};
for(int idx : indexes){
    if( idx >= 0 && idx < arr.length)

        System.out.println("Value at index " + idx + " = " + arr[idx] );
    else
        System.out.println("Index " + idx + " is out of range ! ");
}
}
```

```

public class FindindexVal {
    public static void main (String[] args){
        int[] arr = {34, 10, 66, 27, 47, 8, 55, 78};
        int [] searchValues = {8,55,27};

        System.out.println("(A) Index numbers: " );
        for (int value : searchValues){
            int index = -1;
            for (int i = 0 ; i < arr.length; i++){
                if(arr[i] == value){
                    index = i;
                }
            }
            System.out.println(value + " found at index " + index);
        }

        System.out.println("(B) Values at given indexes:");
        for (int i : searchValues){
            int index = -1;
            for (int j = 0 ; j < arr.length; j++){
                if(arr[j] == i){
                    index = j;
                }
            }
            System.out.println(i + " at index " + index + " = " + arr[index]);
        }
    }
}

```

Run FindindexVal

```

C:\Users\Areeba\.jdk\valhalla-ea-23-valhalla-1-90\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2.1\lib\idea_rt.jar=51
(A) Index numbers:
Value 8 found at index 5
Value 55 found at index 6
Value 27 found at index 3

(B) Values at given indexes:
Value at index 4 = 47
Value at index 6 = 55

Process finished with exit code 0

```

Q5: Write a code to count the total number of duplicate elements in an array

```

public class CountDuplicates {
    public static void main (String [] args){
        int[] arr = {3, 5, 7, 3, 2, 5, 8, 3, 7,};
        int n = arr.length;
        int count = 0 ;

        boolean[] visited = new boolean[n];
        for (int i = 0 ; i < n; i++){
            if (visited[i] ) continue;

            int duplicateCount = 0 ;

            for(int j = i + 1; j < n ; j++)
            {
                if (arr [ i] == arr[j]){
                    visited [j] = true ;
                    duplicateCount++;
                }
            }
            if (duplicateCount > 0 ){
                count ++;
            }
        }
        System.out.println("Total number of duplicate elements: " + count);
    }
}

```

The screenshot shows the IntelliJ IDEA IDE with a Java project named 'DSA'. The file explorer on the left shows the project structure. The main editor displays the code for 'CountDuplicates.java'. The code defines a class 'CountDuplicates' with a static method 'main' that takes an array of integers. It initializes an array 'arr' with the values {3, 5, 7, 3, 2, 5, 8, 3, 7}, calculates its length 'n', and initializes a 'count' variable to 0. It then creates a 'visited' boolean array of size 'n'. A for loop iterates from 'i = 0' to 'i < n'. Inside the loop, it checks if 'visited[i]' is true; if so, it continues the loop. Otherwise, it increments 'count' and marks 'visited[i]' as true. After the loop, it prints the 'duplicateCount' (which is 'count'). The 'Run' tab at the bottom shows the execution output: 'Total number of duplicate elements: 3' and 'Process finished with exit code 0'. The status bar at the bottom indicates the current file is 'CountDuplicates.java' in the 'main' method, with settings for CRLF line endings, UTF-8 encoding, and 4 spaces for indentation.

```
1 public class CountDuplicates {
2     public static void main (String [] args){
3         int[] arr = {3, 5, 7, 3, 2, 5, 8, 3, 7};
4         int n = arr.length;
5         int count = 0 ;
6
7         boolean[] visited = new boolean[n];
8         for (int i = 0 ; i < n; i++){
9             if (visited[i] ) continue;
10
11             int duplicateCount = 0 ;
12         }
13     }
14 }
```

Run CountDuplicates x

C:\Users\Areeba\.jdk\valhalla-ea-23-valhalla+1-90\bin\java.exe --javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2.1\lib\idea_rt.jar=51
Total number of duplicate elements: 3
Process finished with exit code 0

DSA > src > CountDuplicates > main 7:44 CRLF UTF-8 4 spaces